

Impacto de las Bases de Datos Vectoriales en la Recuperación de Información para Agentes de IA Especializados en Documentación de API SDN

Carlos Sebastian Madrigal Rodriguez

July 7, 2026

1 Introducción

Las redes modernas definidas por software (SDN) exponen numerosas APIs para configuración, monitoreo y gestión operativa. Sin embargo, el volumen y la complejidad de la documentación técnica dificultan que los ingenieros y sistemas de automatización identifiquen eficientemente las APIs apropiadas.

Las arquitecturas de Generación Aumentada por Recuperación (RAG, por sus siglas en inglés) ofrecen un enfoque prometedor para este desafío al combinar grandes modelos de lenguaje con fuentes de conocimiento externas. Mediante la recuperación semántica, estos sistemas pueden acceder y razonar sobre documentación técnica en tiempo real, permitiendo que los agentes de IA descubran APIs relevantes y apoyen flujos de trabajo de automatización más allá de la búsqueda tradicional basada en palabras clave (Lewis et al., 2020; Wu et al., 2024).

Las bases de datos vectoriales constituyen un componente clave de las arquitecturas RAG al permitir búsquedas de similitud eficientes sobre representaciones vectoriales de la documentación. Esta capacidad puede mejorar el descubrimiento de información técnica relevante y facilitar el razonamiento contextual sobre grandes repositorios de APIs (Johnson et al., 2019).

A pesar de su creciente adopción, la efectividad de las bases de datos vectoriales para la recuperación de documentación técnica en la automatización de redes sigue siendo insuficientemente comprendida. Por lo tanto,

esta investigación examina el uso de bases de datos vectoriales para la recuperación impulsada por IA de documentación de API SDN y evalúa el rendimiento de bases de datos líderes en la industria como FAISS, PGVector y Neo4j en términos de precisión de recuperación, calidad de respuesta y rendimiento operativo.

2 Materiales y Métodos

2.1 Hipótesis

Hipótesis 1 (Principal): La incorporación de bases de datos vectoriales en agentes de IA basados en RAG mejora la recuperación de conocimiento y el descubrimiento de APIs en documentación SDN en comparación con los métodos de búsqueda tradicionales.

Hipótesis 2: La recuperación basada en bases de datos vectoriales logra mayor precisión de recuperación que la búsqueda basada en palabras clave.

Hipótesis 3: Las implementaciones de bases de datos vectoriales como FAISS, PGVector y Neo4j exhiben diferencias estadísticamente significativas en precisión de recuperación, calidad de respuesta y latencia.

Hipótesis Nula: La elección de la base de datos vectorial no afecta significativamente la calidad de las respuestas generadas por los agentes de IA basados en RAG.

2.2 Adquisición de Datos y Consideraciones Éticas

El corpus experimental fue construido a partir de la documentación de API DevNet de Cisco SD-WAN disponible públicamente (Cisco Systems, Inc., 2025a) y la especificación OpenAPI proporcionada por Cisco SD-WAN vManage (Cisco Systems, Inc., 2025b).

El contenido se procesa únicamente para apoyar los experimentos de recuperación; los documentos originales permanecen como propiedad de sus respectivos dueños y no se redistribuyen. Todas las fuentes se referencian para garantizar transparencia y reproducibilidad.

2.3 Reconocimiento y Preprocesamiento de Datos

Los datos de API en formato JSON que proporciona la especificación OpenAPI de Cisco SD-WAN vManage (Cisco Systems, Inc., 2025b) se analizan para extraer los campos relevantes para la indexación y recuperación. Los principales campos extraídos se resumen en la Tabla 1.

Campo	Significado
path	URL de la API
method	GET/POST/PUT/DELETE
tags	Categoría funcional
summary	Descripción breve
description	Descripción detallada
operationId	Nombre único de operación
parameters	Parámetros de entrada
requestBody	Carga útil de datos enviada
responses	Definición de salida

Table 1: Estructura principal de datos de la API.

Como tarea de preprocesamiento previa a la vectorización, los pares clave-valor se aíslan y convierten en cadenas de lenguaje natural. Por ejemplo, el endpoint de API para recuperar información de dispositivos podría transformarse en una cadena como la siguiente:

Ruta: Administration - Audit Log Endpoint: GET /auditlog/page

Operación: getStatBulkRawPropertyData

Descripción: Obtener datos de propiedades en bruto de forma masiva.

Parámetros: query (requerido), scrollId (opcional), count (requerido).

Retorna: Datos del registro de auditoría.

2.4 Tratamiento de Datos

Los datos serán procesados a través de un pipeline (flujo secuencial) de ingestión común seguido de tres backends independientes de almacenamiento y recuperación. El pipeline de ingestión incluirá extracción de texto, limpieza y generación de representaciones vectoriales utilizando un modelo consistente (Qwen3-Embedding-4B, diseñado específicamente para tareas de incrustación y clasificación de texto) en todas las bases de datos para garan-

tizar la comparabilidad. Cada base de datos se configurará siguiendo las mejores prácticas para un rendimiento y precisión de recuperación óptimos. La arquitectura general del pipeline de ingestión se ilustra en la Figura 1.

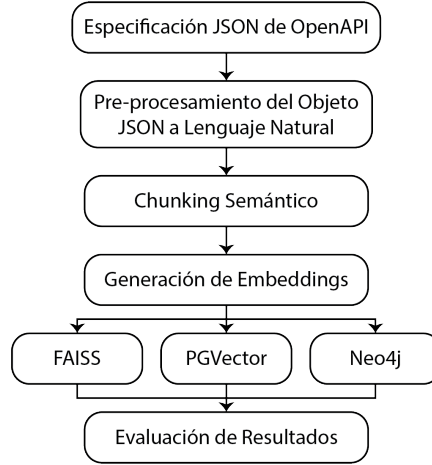


Figure 1: Pipeline de ingestión para el procesamiento de datos de API.

2.5 Métricas de Evaluación

Los resultados de recuperación serán evaluados frente a un conjunto pre-definido de consultas diseñadas para probar diversos aspectos del descubrimiento de APIs y la recuperación de información. Las consultas se subdividen en 3 grupos y se categorizan según su complejidad y relevancia para tareas típicas de automatización de redes:

- **Grupo 1: Descubrimiento Básico de APIs:** Consultas que requieren identificar endpoints o parámetros de API específicos.
- **Grupo 2: Comprensión Contextual:** Consultas que requieren comprender el contexto del uso de la API, como dependencias o casos de uso comunes.
- **Grupo 3: Razonamiento Complejo:** Consultas que requieren razonamiento en múltiples pasos o síntesis de información a través de varias secciones de documentación.

La evaluación se realizará de forma directa con cada base de datos vectorial: consulta semántica, recuperación de documentos y cálculo de métricas sobre el ranking devuelto.

Cada grupo contiene 15 consultas (45 en total). La precisión de recuperación se evaluará con cuatro métricas:

- **Precisión@K**: Fracción de documentos relevantes en los K primeros resultados: $\frac{\text{relevantes en top-}K}{K}$
- **Recall@K**: Fracción de documentos relevantes recuperados sobre el total relevante: $\frac{\text{relevantes en top-}K}{\text{total relevantes}}$
- **MRR**: Promedio de los rangos recíprocos del primer documento relevante por consulta: $\frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$
- **F1@K**: Media armónica de Precisión@K y Recall@K: $2 \cdot \frac{P@K \cdot R@K}{P@K + R@K}$

La calidad de respuestas se evaluará con escala Likert y la latencia se medirá desde el envío de la consulta hasta la respuesta, con temporización automatizada.

Referencias

- Cisco Systems, Inc. (2025a). *Cisco sd-wan api documentation*. <https://developer.cisco.com/docs/sdwan/overview/>
- Cisco Systems, Inc. (2025b). Cisco sd-wan vmanage openapi specification [OpenAPI specification exported from Cisco SD-WAN vManage Release 20.18.2]. Retrieved June 7, 2026, from <https://apidocs/vmanageapi.json>
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3), 535–547.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*. <https://arxiv.org/abs/2005.11401>
- Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T.-W., Guan, N., & Xue, C. J. (2024). Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193*. <https://arxiv.org/abs/2407.13193>